TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ
TRUYỀN THÔNG VIỆT HÀN
KHOA KỸ THUẬT MÁY TÍNH VÀ ĐIỆN TỬ



ĐỒ ÁN CHUYÊN NGÀNH 2

CUSTOM YOCTO BSP AND OUT-OF-TREE KERNEL DRIVER FOR THE MPU6050 SENSOR ON BEAGLEBONE BLACK

Họ và tên sinh viên: *Nguyễn Minh Anh Tú*
Mã sinh viên : *23CE085*
Lớp: *23ES*
Ngành: *Hệ thống nhúng và IoT*
Người hướng dẫn: *TS. Dương Ngọc Pháp*

Đà Nẵng, tháng 11 năm 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ
TRUYỀN THÔNG VIỆT HÀN
KHOA KỸ THUẬT MÁY TÍNH VÀ ĐIỆN TỬ



ĐỒ ÁN CHUYÊN NGÀNH 2

CUSTOM YOCTO BSP AND OUT-OF-TREE KERNEL DRIVER FOR THE MPU6050 SENSOR ON BEAGLEBONE BLACK

Họ và tên sinh viên: *Nguyễn Minh Anh Tú*
Mã sinh viên : *23CE085*
Lớp: *23ES*
Ngành: *Hệ thống nhúng và IoT*
Người hướng dẫn: *TS. Dương Ngọc Pháp*

Đà Nẵng, tháng 11 năm 2026

LỜI CẢM ƠN

Em xin cảm ơn thầy TS. Dương Ngọc Pháp đã hướng dẫn em trong suốt quá trình làm đồ án, từ lúc chọn đề tài, góp ý đề cương đến khi hoàn thành sản phẩm. Những góp ý của thầy giúp em giới hạn phạm vi cho vừa thời gian và tập trung vào phần cốt lõi là BSP và driver.

Em cũng cảm ơn các thầy cô Khoa Kỹ thuật Máy tính và Điện tử đã trang bị kiến thức nền về hệ thống nhúng, hệ điều hành và mạch điện tử, và cảm ơn gia đình, bạn bè đã động viên em trong thời gian thực hiện đề tài.

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

……

……

……

……

……

……

……

……

……

……

……

……

……

Đà Nẵng, .../.../2026

Giảng viên hướng dẫn

TS. Dương Ngọc Pháp

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT ..... **vi**
DANH MỤC HÌNH ẢNH ..... **viii**
DANH MỤC BẢNG ..... **ix**
MỞ ĐẦU **1**
1. Lý do chọn đề tài 1
2. Mục tiêu nghiên cứu 1
3. Đối tượng và phạm vi nghiên cứu 1
4. Phương pháp nghiên cứu 2
5. Ý nghĩa khoa học và thực tiễn của đề tài 2
6. Bố cục của báo cáo 3
CHƯƠNG 1. TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT **4**
1.1. Hệ thống Linux nhúng và BSP 4
1.1.1. Thành phần của một hệ thống Linux nhúng *4*
1.1.2. BeagleBone Black và SoC AM335x *4*
1.1.3. Quá trình khởi động trên AM335x *4*
1.2. Yocto Project 5
1.2.1. BitBake, recipe, layer và image *5*
1.2.2. Kernel trong Yocto *5*
1.3. Device tree và overlay 5
1.4. Mô hình driver trong nhân Linux 5
1.4.1. Bus, device, driver và probe; hệ thống con I2C *5*
1.4.2. Char device, misc device và uapi *6*
1.4.3. sysfs và debugfs *6*
1.4.4. Xử lý ngắt: nửa trên và threaded IRQ *6*
1.4.5. Đồng bộ và vòng đời đối tượng *6*
1.5. PREEMPT_RT 6
1.6. Cảm biến MPU-6500 / MPU6050 7
1.6.1. Đặc điểm và thanh ghi *7*
1.6.2. Khác biệt giữa MPU-6050 và MPU-6500 ..... *7*
1.6.3. Giao thức I2C *8*
1.7. Driver inv_mpu6050 của mainline và hướng tiếp cận của đề tài 8
CHƯƠNG 2. XÂY DỰNG BSP BẰNG YOCTO **10**
2.1. Phần cứng và môi trường phát triển 10
2.1.1. Sơ đồ kết nối *10*
2.1.2. Môi trường build và công cụ *11*

2.2. Layer meta-projectb 11
2.3. Image projectb-image 12
2.4. Bootloader, extlinux và chuỗi khởi động 12
2.5. Kernel: linux-yocto, linux-yocto-rt và kernel debug 13
2.6. Device tree overlay cho MPU6050 13
2.7. Ghi thẻ và kiểm chứng bring-up 14
2.8. Quy trình phát triển driver 14
CHƯƠNG 3. THIẾT KẾ, HIỆN THỰC VÀ KIỂM THỬ DRIVER MPU6050 16
3.1. Yêu cầu và hợp đồng giao diện 16
3.1.1. Yêu cầu 16
3.1.2. Header uapi 16
3.1.3. Thuộc tính sysfs 17
3.2. Kiến trúc driver 17
3.2.1. Probe, reset và cấu hình chip 18
3.2.2. Char device 18
3.2.3. Nguồn dữ liệu qua các bước 19
3.3. Ngắt DATA_RDY và đọc FIFO theo lô 20
3.3.1. Nửa trên và hàng đợi cạnh 20
3.3.2. Thread ngắt: ghép khung với cạnh 21
3.3.3. Cỡ lô theo tốc độ bus và hiện tượng trượt con trỏ FIFO 21
3.3.4. Chờ sau cạnh INT 22
3.3.5. Tràn FIFO 23
3.4. Đồng bộ và khóa 23
3.5. Đường lỗi và vòng đời 24
3.5.1. Gỡ thiết bị khi ứng dụng còn mở 24
3.5.2. Lỗi bus I2C và chip mất nguồn 24
3.6. Tích hợp vào image 25
3.7. Kiểm thử và đánh giá 26
3.7.1. Phương pháp và công cụ 26
3.7.2. Kết quả trên kernel nộp 26
3.7.3. Đường lỗi 27
3.7.4. Demo 28
3.7.5. Hạn chế 28
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 30
Kết luận 30

Hướng phát triển 30
DANH MỤC TÀI LIỆU THAM KHẢO 31
PHỤ LỤC 32
Phụ lục A. Build lại image và chạy kiểm thử 32
Phụ lục B. Cấu trúc mã nguồn 32
Phụ lục C. Tiến độ thực hiện thực tế.....32

DANH MỤC TỪ VIẾT TẮT

TỪ VIẾT TẮT	Ý NGHĨA
BBB	BeagleBone Black
BSP	Board Support Package – gói hỗ trợ board
DLPF	Digital Low-Pass Filter – bộ lọc thông thấp số
DT	Device Tree – cây thiết bị
FIFO	First In First Out – bộ đệm vào trước ra trước
GPIO	General Purpose Input/Output
I2C	Inter-Integrated Circuit – bus nối tiếp hai dây
IIO	Industrial I/O – hệ thống con cảm biến của nhân Linux
IRQ	Interrupt Request – yêu cầu ngắt
LA	Logic Analyzer – máy phân tích logic
LSB	Least Significant Bit – bit có trọng số nhỏ nhất
MDIO	Management Data Input/Output – bus quản lý PHY Ethernet
MLO	MMC Loader – tên tệp SPL trên AM335x
PHY	Physical layer – chip lớp vật lý Ethernet
PLL	Phase-Locked Loop – vòng khóa pha
PREEMPT_RT	Bản vá thời gian thực của nhân Linux
SMBus	System Management Bus – giao thức con của I2C
SoC	System on Chip
SPL	Secondary Program Loader – bootloader tầng đầu
sstate	Shared state cache của BitBake

uapi	User-space API – giao diện nhân dành cho ứng dụng
WIC	OpenEmbedded Image Creator – công cụ tạo ảnh thẻ nhớ

DANH MỤC HÌNH ẢNH

Hình 2.1. Sơ đồ kết nối BeagleBone Black và module MPU6050 10
Hình 2.2. Chuỗi khởi động của image nộp.....12
Hình 2.3. Logic analyzer: đọc thanh ghi WHO_AM_I, chip trả 0x70.....14
Hình 3.1. Các tầng của hệ thống và giao diện giữa các tầng.....18
Hình 3.2. Logic analyzer: một giao dịch burst 14 byte từ thanh ghi 0x3B 18
Hình 3.3. Luồng dữ liệu trong driver 20
Hình 3.4. Logic analyzer: INT và đọc FIFO ở 100 Hz, I2C 400 kHz.....21
Hình 3.5. Logic analyzer: giao dịch bắt đầu trong xung INT bị chip NACK 22
Hình 3.6. Rút SDA khi đang đọc ở 100 Hz: bản ghi mỗi giây và dropped.....28
Hình 3.7. Gia tốc ba trục khi nghiêng module trong buổi diễn tập demo 28

DANH MỤC BẢNG

Bảng 1.1. Thông số BeagleBone Black liên quan đến đề tài 4
Bảng 1.2. Thanh ghi MPU-6500 được driver sử dụng....7
Bảng 1.3. Khác biệt MPU-6050 và MPU-6500 liên quan đến driver 8
Bảng 1.4. So sánh driver inv_mpu6050 (IIO) và driver của đề tài 9
Bảng 2.1. Bảng nối chân....10
Bảng 2.2. Phiên bản phần mềm đã chốt 11
Bảng 2.3. Các recipe chính của layer meta-projectb....11
Bảng 3.1. Hợp đồng /dev/mpu6050....17
Bảng 3.2. Thuộc tính sysfs trong /sys/class/misc/mpu6050/....17
Bảng 3.3. So sánh ba cách lấy dữ liệu....19
Bảng 3.4. Cỡ lô theo tốc độ lấy mẫu ở I2C 400 kHz 22
Bảng 3.5. Đọc FIFO_COUNT trên kernel nộp trước và sau khi sửa (ftrace, 30000 bản ghi)....23
Bảng 3.6. Dữ liệu chia sẻ trong driver....24
Bảng 3.7. Bộ kiểm thử....26
Bảng 3.8. Kết quả kiểm thử trên kernel nộp....27
Bảng 3.9. Kết quả kiểm thử đường lỗi 27
Bảng PL.1. Tiến độ thực hiện thực tế....32

MỞ ĐẦU

1. Lý do chọn đề tài

Linux nhúng hiện được dùng rộng rãi trong thiết bị công nghiệp, thiết bị mạng, thiết bị y tế và robot. Muốn chạy Linux trên một phần cứng cụ thể, người phát triển cần xây dựng gói hỗ trợ board (BSP) gồm bootloader, kernel, device tree và hệ thống tệp gốc, sau đó viết driver cho những ngoại vi mà nhân chưa hỗ trợ hoặc hỗ trợ chưa phù hợp với yêu cầu.

Trong nhiều tài liệu học tập, cảm biến được đọc bằng cách mở /dev/i2c-N từ không gian người dùng. Cách làm này nhanh nhưng bỏ qua mô hình driver của nhân, gồm device tree, hàm probe, xử lý ngắt, đồng bộ và quản lý vòng đời đối tượng. Đây lại là những nội dung khó nắm vững nhất và thường được yêu cầu ở vị trí kỹ sư Embedded Linux/BSP.

Vì vậy, đề tài chọn BeagleBone Black (SoC AM335x, dòng Sitara của Texas Instruments) và module cảm biến gia tốc MPU6050 (giao tiếp I2C, có chân ngắt và FIFO) để thực hiện trọn vẹn quy trình: tự xây dựng bản phân phối bằng Yocto Project, viết driver nhân ngoài cây (out-of-tree) sử dụng ngắt và FIFO của chip, tích hợp driver vào image và kiểm chứng bằng phép đo trên phần cứng thật.

2. Mục tiêu nghiên cứu

Mục tiêu chung là xây dựng một hệ thống Linux nhúng tự build bằng Yocto chạy trên BeagleBone Black, kèm driver nhân tự viết cho MPU6050, dùng ngắt và FIFO của chip, tự nạp khi khởi động, hoạt động ổn định cả khi có lỗi phần cứng và có bằng chứng kiểm chứng. Các nhiệm vụ cụ thể gồm:

- Nghiên cứu luồng khởi động AM335x và kiến trúc Yocto; xây layer BSP riêng, image tùy biến, kernel PREEMPT_RT và kernel thường.
- Nghiên cứu mô hình driver Linux, hệ thống con I2C, device tree, threaded IRQ và đồng bộ.
- Thiết kế và hiện thực driver: char device /dev/mpu6050 (read, poll, ioctl) trả bản ghi nhị phân có timestamp, header uapi, sysfs cho cấu hình, ngắt DATA_RDY kết hợp đọc FIFO theo lô.
- Xử lý đường lỗi: lỗi bus I2C, chip mất nguồn, tràn FIFO, gỡ thiết bị khi đang đọc.
- Tích hợp driver và chương trình kiểm thử vào image; kiểm chứng bằng logic analyzer, ftrace, bộ kiểm thử tự động và tải nền.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng là BSP Yocto cho BeagleBone Black và driver nhân cho module MPU6050. Kiểm tra WHO_AM_I cho thấy chip thật trên module thuộc họ MPU-6500 (giá trị 0x70), driver hỗ trợ cả hai họ và chọn thông số theo WHO_AM_I lúc probe.

- Phần cứng: BeagleBone Black (AM3358, một nhân), module MPU6050 trên I2C2 (400 kHz), chân INT nối GPIO1_28.
- Phần mềm: Yocto Project 5.2.4 (Walnascar), nhân Linux 6.12.47, U-Boot 2025.01, systemd.
- Driver: gia tốc ba trục và nhiệt độ qua FIFO, 4–1000 Hz, dải ±2/4/8/16 g; không xuất dữ liệu con quay hồi chuyển.
- Kiểm thử trên kernel PREEMPT_RT (bản nộp) và kernel thường, ở mức chức năng và độ ổn định.
- Ngoài phạm vi: benchmark độ trễ thời gian thực (cyclictest), giao diện đồ họa, cập nhật OTA, driver inv_mpu6050 có sẵn của mainline.

4. Phương pháp nghiên cứu

- Nghiên cứu tài liệu: datasheet và register map MPU-6500/MPU-6050, TRM AM335x, tài liệu Yocto và tài liệu nhân; đối chiếu với driver inv_mpu6050 của mainline.
- Phát triển lặp trên phần cứng thật: mỗi tuần thêm một tầng chức năng (I2C, char device, sysfs, ngắt, FIFO, đường lỗi) kèm mốc kiểm cụ thể; phát triển trên kernel debug có lockdep.
- Đối chứng đầu–cuối: byte giải mã từ logic analyzer phải trùng byte ứng dụng đọc được; ftrace đo thời gian giữa ngắt, timestamp và giao dịch I2C.
- Kiểm thử định lượng: bảo toàn số ngắt = số bản ghi + số mẫu mất, nội dung dữ liệu ($|a| \approx 1$ g, nhiệt độ hợp lý), tải nền stress-ng, gây lỗi có chủ đích (rút dây, tràn FIFO, unbind).
- Quản lý mã nguồn bằng Git, gắn tag cho mốc bring-up và mốc nộp.

5. Ý nghĩa khoa học và thực tiễn của đề tài

Về mặt khoa học, đề tài trình bày đầy đủ các tầng của một hệ thống Linux nhúng, từ phần cứng tới ứng dụng, kèm số liệu đo trên phần cứng thật. Báo cáo cũng ghi nhận một số hiện tượng không được mô tả trong datasheet: chip trượt con trỏ FIFO khi bị đọc đúng lúc đang ghi, chip từ chối giao dịch I2C trong thời gian phát xung ngắt, và độ trễ ngắt khác nhau giữa kernel debug và kernel nộp.

Về mặt thực tiễn, sản phẩm có thể làm nền tảng cho thiết bị giám sát rung động và tình trạng máy (condition monitoring). Giao diện /dev/mpu6050 đã được cố định và sẽ giữ nguyên ở giai đoạn 2 (middleware sensord, giao diện người dùng, đo độ trễ thời gian thực).

6. Bố cục của báo cáo

Ngoài phần Mở đầu, Kết luận, Tài liệu tham khảo và Phụ lục, báo cáo gồm ba chương:

- Chương 1 trình bày tổng quan và cơ sở lý thuyết: hệ thống Linux nhúng và BSP, Yocto Project, device tree, mô hình driver của nhân Linux, PREEMPT_RT và cảm biến MPU-6500/MPU6050.
- Chương 2 trình bày quá trình xây dựng BSP bằng Yocto: phần cứng, layer meta-projectb, image, bootloader, kernel, device tree overlay, ghi thẻ và kiểm chứng bring-up.
- Chương 3 trình bày thiết kế, hiện thực và kiểm thử driver MPU6050: giao diện, kiến trúc, xử lý ngắt và FIFO, đồng bộ, đường lỗi, tích hợp vào image và kết quả kiểm thử.

CHƯƠNG 1. TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT

1.1. HỆ THỐNG LINUX NHÚNG VÀ BSP

1.1.1. Thành phần của một hệ thống Linux nhúng

Một hệ thống Linux nhúng gồm bốn phần chạy nối tiếp nhau khi khởi động: bootloader khởi tạo bộ nhớ và nạp kernel; kernel quản lý phần cứng qua driver; device tree mô tả phần cứng không tự dò được (địa chỉ thanh ghi, ngắt, chân) để cùng một kernel chạy trên nhiều board; hệ thống tệp gốc (rootfs) chứa init, thư viện và ứng dụng. BSP là phần riêng của một board: cấu hình bootloader, cấu hình và bản vá kernel, device tree, cùng driver cho thiết bị trên board.

1.1.2. BeagleBone Black và SoC AM335x

BeagleBone Black dùng SoC AM3358 của Texas Instruments. Thông số chính liên quan đến đề tài nêu ở Bảng 1.1. Khi thiết kế driver cần lưu ý SoC chỉ có một nhân CPU. Nửa trên của ngắt, thread ngắt, ứng dụng và tải nền cùng chia nhau một nhân, nên thứ tự ưu tiên và độ dài các vùng cấm ngắt ảnh hưởng trực tiếp tới độ trễ.

Bảng 1.1. Thông số BeagleBone Black liên quan đến đề tài

Thành phần	Thông số
CPU	ARM Cortex-A8 1 GHz, một nhân (AM3358)
Bộ nhớ	512 MB DDR3, eMMC 4 GB (không dùng), khe microSD
I2C	I2C0 (PMIC, HDMI), I2C2 ra header P9_19/P9_20
GPIO	4 bank, mỗi bank một ngắt tới INTC; chân P9_12 là GPIO1_28
Kết nối	Ethernet 10/100 (PHY LAN8710), UART0 console, HDMI (TDA19988)

1.1.3. Quá trình khởi động trên AM335x

Khi cấp nguồn, ROM code trong SoC tìm bootloader theo thứ tự chân SYSBOOT (eMMC trước, nút S2 đổi sang thẻ SD), nạp SPL (tệp MLO) vào SRAM nội. SPL khởi tạo DDR rồi nạp U-Boot đầy đủ. U-Boot đọc extlinux.conf trên phân vùng boot, nạp zImage, device tree và overlay, ghép overlay vào device tree rồi nhảy vào kernel với cmdline lấy từ dòng append. Kernel gắn rootfs và chạy init (systemd). Chuỗi cụ thể của image đề tài trình bày ở mục 2.4.

1.2. YOCTO PROJECT

1.2.1. BitBake, recipe, layer và image

Yocto Project là bộ công cụ để tạo bản phân phối Linux tùy biến từ mã nguồn. BitBake là trình thực thi tác vụ: mỗi phần mềm được mô tả bằng một recipe (.bb) gồm nguồn (SRC_URI), cách cấu hình, biên dịch, cài đặt và đóng gói. Recipe được nhóm thành layer; layer sau có thể sửa recipe của layer trước bằng tệp .bbappend mà không đụng vào mã gốc. Image là một recipe đặc biệt liệt kê các gói cần cài vào rootfs.

Ba biến cấu hình quyết định kết quả build: MACHINE (board đích, ở đây beaglebone-yocto), DISTRO (chính sách bản phân phối, ở đây poky) và các biến trong local.conf (init system, kernel nào, số luồng). Kết quả mỗi tác vụ được lưu vào shared state cache (sstate) theo chữ ký của toàn bộ đầu vào; build lại chỉ chạy những tác vụ có đầu vào thay đổi.

1.2.2. Kernel trong Yocto

Recipe linux-yocto lấy nhân từ kho của Yocto kèm kernel-cache, nơi chứa cấu hình theo board và loại kernel (standard, preempt-rt). Cấu hình riêng của dự án được thêm bằng fragment (.cfg) trong .bbappend; Yocto ghép fragment vào cấu hình gốc rồi kiểm từng dòng có được áp hay không. Module ngoài cây dùng lớp module.bbclass, build với nguồn nhân đã cấu hình trong sysroot.

1.3. DEVICE TREE VÀ OVERLAY

Device tree là cấu trúc dạng cây mô tả phần cứng: mỗi node có compatible (chuỗi để ghép với driver), reg (địa chỉ), interrupts, pinctrl. Bus I2C không tự dò được thiết bị, nên thiết bị trên I2C phải khai trong device tree; I2C core đọc node con của bộ điều khiển và tạo i2c_client cho từng node.

Overlay (.dtbo) là phần bổ sung được ghép vào device tree gốc lúc boot, tham chiếu node gốc qua nhãn (&i2c2, &gpio1). Muốn ghép được, device tree gốc phải được biên dịch với cờ -@ để giữ bảng __symbols__. Overlay cho phép thêm cảm biến mà không sửa tệp dts của nhân.

1.4. MÔ HÌNH DRIVER TRONG NHÂN LINUX

1.4.1. Bus, device, driver và probe; hệ thống con I2C

Mô hình driver tách ba khái niệm: bus (I2C, platform...), device (một thiết bị cụ thể trên bus) và driver (mã điều khiển một loại thiết bị). Khi có device và driver khớp nhau (theo bảng of_device_id với compatible), bus gọi hàm probe của driver; khi gỡ, gọi remove. Với I2C, driver khai báo struct i2c_driver, truy cập thanh ghi qua các hàm

SMBus như i2c_smbus_read_byte_data và i2c_smbus_read_i2c_block_data. MODULE_DEVICE_TABLE sinh chuỗi alias để udev tự nạp module khi thiết bị xuất hiện.

1.4.2. Char device, misc device và uapi

Ứng dụng giao tiếp với driver qua tệp thiết bị. Driver cung cấp struct file_operations gồm open, release, read, poll, unlocked_ioctl. Misc device là char device dùng chung major 10, đăng ký bằng misc_register, tự tạo /dev/<tên>. Dữ liệu trao đổi giữa nhân và ứng dụng (struct, mã ioctl) đặt trong header uapi dùng chung, để hai bên không bao giờ lệch bố cục. poll() kết hợp với wait queue cho phép ứng dụng ngủ tới khi có dữ liệu thay vì hỏi vòng.

1.4.3. sysfs và debugfs

sysfs xuất thuộc tính của thiết bị dưới dạng tệp văn bản, mỗi tệp một giá trị, phù hợp cho cấu hình (tốc độ lấy mẫu, dải đo). Tên thuộc tính sysfs là giao diện ổn định với ứng dụng. debugfs dùng cho bộ đếm chẩn đoán, không phải giao diện ổn định.

1.4.4. Xử lý ngắt: nửa trên và threaded IRQ

Hàm xử lý ngắt chạy trong ngữ cảnh không được ngủ, nên không thể gọi giao dịch I2C (phải chờ bus). request_threaded_irq chia việc làm hai: nửa trên chạy ngay khi có ngắt, làm việc ngắn rồi trả IRQ_WAKE_THREAD; thread ngắt (SCHED_FIFO, ưu tiên 50) làm phần còn lại và được phép ngủ. Cờ IRQF_ONESHOT giữ ngắt bị che cho tới khi thread xong. Với ngắt từ GPIO, kiểu kích (cạnh lên, mức) lấy từ device tree.

1.4.5. Đồng bộ và vòng đời đối tượng

Driver có nhiều luồng chạy song song: nửa trên, thread ngắt, các tiến trình gọi read, ghi sysfs, work queue. Các công cụ dùng trong đề tài: mutex cho đoạn được ngủ, biến atomic cho bộ đếm, kfifo một-ghi-một-đọc không cần khóa, wait queue cho read/poll. Lockdep là bộ kiểm tra thứ tự khóa lúc chạy của nhân, báo lỗi khi phát hiện khả năng khóa chết.

Tài nguyên cấp bằng devres (devm_*) được giải phóng tự động sau remove. Cách này phù hợp với ngắt, debugfs, nhưng không phù hợp với dữ liệu mà một tệp đang mở còn dùng sau khi thiết bị bị gỡ. Trường hợp đó cần đếm tham chiếu (kref): đối tượng chỉ được giải phóng khi người giữ cuối cùng trả tham chiếu.

1.5. PREEMPT_RT

PREEMPT_RT là tập bản vá (đã vào mainline từ 6.12) biến nhân thành gần như hoàn toàn chiếm quyền được: spinlock thường thành mutex có kế thừa ưu tiên, phần lớn

ngắt chạy trong thread. Hệ quả cho driver: nửa trên của ngắt GPIO cũng chạy trong thread irq/26 của gpio-omap, nên timestamp lấy trong nửa trên đã gồm độ trễ đánh thức thread đó. Đề tài dùng kernel RT làm kernel nộp, nhưng không đo benchmark thời gian thực (để giai đoạn 2).

1.6. CẢM BIẾN MPU-6500 / MPU6050

1.6.1. Đặc điểm và thanh ghi

MPU-6500 là cảm biến quán tính 6 trục (gia tốc 3 trục, con quay 3 trục, cảm biến nhiệt) với ADC 16 bit, giao tiếp I2C tới 400 kHz. Chip có FIFO 512 byte, chọn được dữ liệu nào ghi vào, và chân INT phát xung khi có mẫu mới (DATA_RDY). Chip không có ngắt "FIFO đủ N mẫu", chỉ có DATA_RDY, tràn FIFO và các ngắt chuyển động. Các thanh ghi driver dùng nêu ở Bảng 1.2.

Bảng 1.2. Thanh ghi MPU-6500 được driver sử dụng

Địa chỉ	Tên	Dùng để
0x19	SMPLRT_DIV	tốc độ = 1000 / (1 + div) Hz
0x1A, 0x1D	CONFIG, ACCEL_CONFIG2	bộ lọc thông thấp (DLPF) cho con quay, gia tốc
0x1B	GYRO_CONFIG	ghi 0: ±250 °/s, qua DLPF
0x1C	ACCEL_CONFIG	dải đo ±2/4/8/16 g
0x23	FIFO_EN	chọn gia tốc + nhiệt vào FIFO (8 byte mỗi mẫu)
0x37, 0x38	INT_PIN_CFG, INT_ENABLE	xung INT 50 µs, bật DATA_RDY
0x3B–0x42	ACCEL_XOUT_H…TEMP_OUT_L	đọc trực tiếp một mẫu (tuần 4–6; bản cuối đọc qua FIFO)
0x68	SIGNAL_PATH_RESET	reset đường tín hiệu sau DEVICE_RESET (họ 6500)
0x6A	USER_CTRL	bật FIFO, reset FIFO
0x6B, 0x6C	PWR_MGMT_1, PWR_MGMT_2	reset chip, chọn nhịp PLL; bật mọi trục
0x72–0x74	FIFO_COUNT, FIFO_R_W	số byte trong FIFO, đọc FIFO
0x75	WHO_AM_I	0x70 (MPU-6500), 0x68 (MPU-6050)

1.6.2. Khác biệt giữa MPU-6050 và MPU-6500

Module bán trên thị trường mang tên MPU6050 nhưng chip trên module của đề tài trả WHO_AM_I = 0x70 và PWR_MGMT_1 = 0x01 sau khi cấp nguồn, đúng giá trị

reset của MPU-6500. Hai họ cùng địa chỉ I2C và gần như cùng bản đồ thanh ghi, khác ở các điểm trong Bảng 1.3. Nếu dùng nhầm công thức nhiệt độ của MPU-6050, nhiệt độ đo trong phòng là khoảng 48 °C; dùng công thức của MPU-6500 thì được 32.7 °C, phù hợp với thực tế.

Bảng 1.3. Khác biệt MPU-6050 và MPU-6500 liên quan đến driver

Mục	MPU-6050	MPU-6500
WHO_AM_I	0x68	0x70
PWR_MGMT_1 sau reset	0x40 (ngủ)	0x01
Kích thước FIFO	1024 byte	512 byte
Nhiệt độ (°C)	TEMP/340 + 36.53	TEMP/333.87 + 21
ACCEL_CONFIG2 (DLPF gia tốc)	không có	có (0x1D)

1.6.3. Giao thức I2C

I2C dùng hai dây kéo lên: SCL (nhịp) và SDA (dữ liệu). Giao dịch bắt đầu bằng START (SDA xuống khi SCL cao), gửi địa chỉ 7 bit kèm bit đọc/ghi, mỗi byte theo sau một bit ACK do bên nhận kéo SDA xuống, kết thúc bằng STOP. Đọc một thanh ghi gồm pha ghi địa chỉ thanh ghi, repeated START rồi pha đọc; byte đọc cuối được master trả NACK để chip nhả bus. Một byte trên dây là 9 nhịp SCL, nên ở 400 kHz mỗi byte mất 22.5 µs; con số này quyết định cỡ lô đọc FIFO ở mục 3.3.

1.7. DRIVER INV_MPU6050 CỦA MAINLINE VÀ HƯỚNG TIẾP CẬN CỦA ĐỀ TÀI

Nhân Linux đã có driver cho họ cảm biến này trong hệ thống con IIO (drivers/iio/imu/inv_mpu6050), hỗ trợ MPU-6050, MPU-6500, MPU-9250 và nhiều chip ICM. IIO chuẩn hóa cảm biến: thuộc tính sysfs in_accel_x_raw, in_accel_scale, sampling_frequency; dữ liệu liên tục qua buffer /dev/iio:deviceN với trigger từ ngắt data ready; driver đọc FIFO trong thread ngắt và ước lượng timestamp cho từng mẫu. Bảng 1.4 so sánh hai hướng.

Bảng 1.4. So sánh driver inv_mpu6050 (IIO) và driver của đề tài

Tiêu chí	inv_mpu6050 (IIO)	Driver đề tài
Giao diện	chuẩn IIO, công cụ có sẵn (iio_readdev, libiio)	/dev/mpu6050 riêng, bản ghi 16 byte cố định
Phạm vi chip	nhiều họ, I2C và SPI, con quay, từ kế	MPU-6050/6500, I2C, gia tốc + nhiệt
Timestamp	ước lượng theo chu kỳ, lọc	lấy theo từng cạnh INT, ghép từng mẫu
Lỗi bus, mất nguồn	theo khung IIO	tự đếm dropped, watchdog khởi tạo lại
Mục đích	sản phẩm, dùng chung	học mô hình driver, kiểm soát từng chi tiết

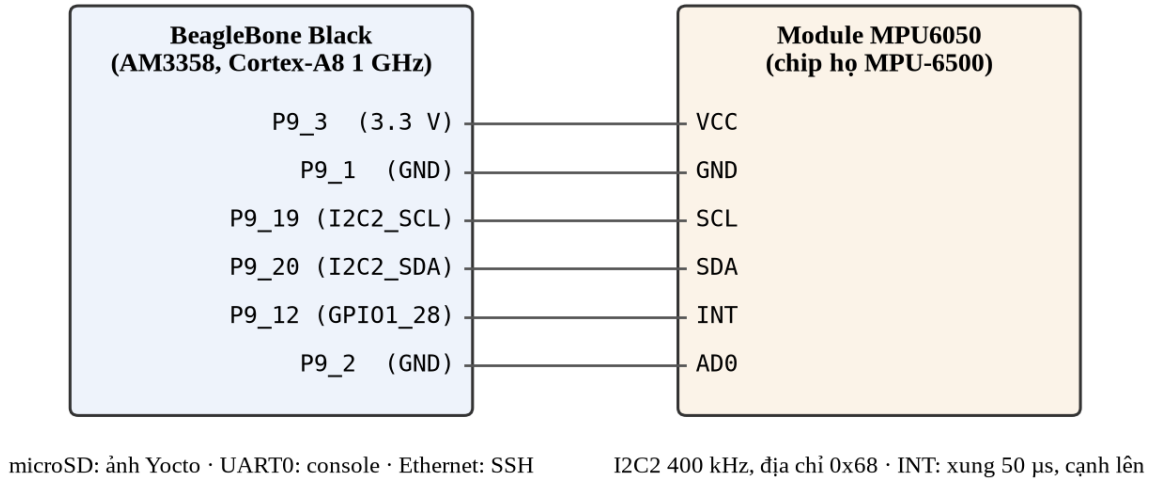
Đề tài không dùng inv_mpu6050 vì mục tiêu là tự hiện thực và hiểu từng tầng. Để driver mainline không bao giờ nhận thiết bị, overlay dùng compatible riêng "projectb,mpu6050" và kernel không bật CONFIG_IIO. Ở giai đoạn 2, nếu sản phẩm cần dùng công cụ IIO, có thể chuyển sang inv_mpu6050 hoặc bọc driver hiện tại bằng giao diện IIO.

CHƯƠNG 2. XÂY DỰNG BSP BẰNG YOCTO

2.1. PHẦN CỨNG VÀ MÔI TRƯỜNG PHÁT TRIỂN

2.1.1. Sơ đồ kết nối

Module MPU6050 nối vào header P9 của BeagleBone Black như Hình 2.1 và Bảng 2.1. AD0 nối đất nên địa chỉ I2C là 0x68. Chân SDA, SCL đã được device tree gốc đặt ở mode 3 (I2C2) có kéo lên nội, đọc thanh ghi pad bằng devmem ra 0x33. Chân INT được cấu hình là đầu vào có kéo xuống, chip đẩy kéo (push-pull) mức cao 50 µs mỗi mẫu.



Hình 2.1. Sơ đồ kết nối BeagleBone Black và module MPU6050

Bảng 2.1. Bảng nối chân

Chân BBB	Chức năng	Module
P9_3	3.3 V	VCC
P9_1	GND	GND
P9_19	I2C2_SCL	SCL
P9_20	I2C2_SDA	SDA
P9_12	GPIO1_28, ngắt cạnh lên	INT
P9_2	GND	AD0

Ở lần nối dây đầu tiên, i2cdetect không phát hiện được thiết bị dù điện áp tĩnh trên các chân đều đúng. Nguyên nhân là dây SDA bị cắm nhầm chân, lỗi này chỉ lộ ra khi có giao dịch trên bus. Vì vậy, từ đó về sau mọi thay đổi về nối dây đều được kiểm tra lại bằng logic analyzer.

2.1.2. Môi trường build và công cụ

Yocto được build trong container Docker Ubuntu 24.04 (user trùng uid với máy chủ) để môi trường build độc lập với hệ điều hành máy chủ. Phiên bản các thành phần nêu ở Bảng 2.2. Công cụ đo gồm: cáp USB–UART cho console, logic analyzer 8 kênh (sigrok/PulseView, 2–24 MHz) cho SCL, SDA, INT; ftrace của nhân để đo thời gian trên board.

Bảng 2.2. Phiên bản phần mềm đã chốt

Thành phần	Phiên bản
Yocto Project / Poky	5.2.4 "Walnascar"
Nhân Linux	linux-yocto 6.12.47 (standard và preempt-rt)
Bootloader	U-Boot 2025.01
Init	systemd 257
MACHINE / DISTRO	beaglebone-yocto / poky
Môi trường build	Docker Ubuntu 24.04, 3 luồng BitBake

2.2. LAYER META-PROJECTB

Mọi tùy biến của dự án nằm trong một layer riêng meta-projectb, không sửa tệp nào của Poky. Cấu hình build (local.conf, bblayers.conf) sinh từ mẫu trong repo bằng script setup-build.sh, để build lại từ đầu cho ra cùng kết quả. Bảng 2.3 liệt kê các recipe chính.

Bảng 2.3. Các recipe chính của layer meta-projectb

Recipe / tệp	Vai trò
recipes-core/images/projectb-image.bb	image: core-image-minimal + SSH, công cụ, driver, tự kiểm
wic/projectb.wks, projectb-extlinux.conf	bố cục thẻ SD, cmdline và dòng nạp overlay
recipes-kernel/linux/*.bbappend, *.cfg	kernel: dtb với -@, hid-multitouch, kernel debug
recipes-bsp/projectb-overlays	biên dịch mpu6050.dts thành mpu6050.dtbo
recipes-kernel/mpu6050/mpu6050-mod	module ngoài cây, nguồn ở driver/mpu6050
recipes-kernel/mpu6050/mpu6050-uapi	header uapi cho ứng dụng (allarch)
recipes-apps/mpu6050-test	chương trình kiểm thử /dev/mpu6050
recipes-apps/mpu6050-boot-check	rule udev, unit systemd tự kiểm khi khởi động
recipes-core/projectb-ssh-keys, projectb-phy-retry	khóa SSH; gắn lại PHY Ethernet

Header uapi tách thành recipe riêng kiểu allarch, không gắn với recipe module. Recipe module phụ thuộc kernel nên mỗi lần đổi kernel (RT, thường, debug) phải build lại; nếu ứng dụng lấy header từ recipe module thì cũng phải build lại theo. Tách ra, ứng dụng chỉ phụ thuộc vào header.

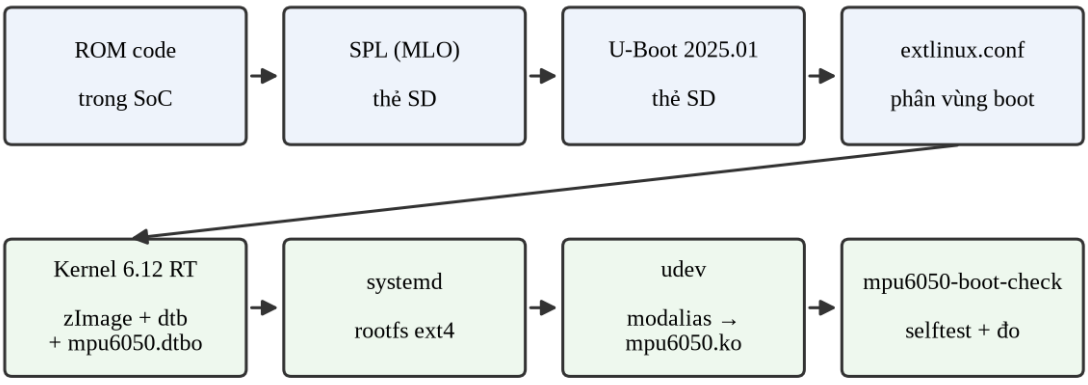
2.3. IMAGE PROJECTB-IMAGE

Image kế thừa core-image-minimal, thêm máy chủ SSH (image feature ssh-server-openssh), i2c-tools, stress-ng, rt-tests, gdbserver, cùng các gói của dự án. Từ tuần 2, init là systemd thay cho sysvinit mặc định. Đăng nhập SSH chỉ bằng khóa: khóa công khai được cài vào authorized_keys bằng recipe projectb-ssh-keys. Kiểm tra cho thấy tùy chọn allow-empty-password khiến SSH đăng nhập được không cần khóa, nên tùy chọn này đã được bỏ; phép thử âm với mật khẩu rỗng trả Permission denied.

PHY Ethernet LAN8710 của board không trả lời MDIO ở lần probe đầu tiên (lỗi -5) trong mọi lần khởi động; có lần kernel gắn lại được khi mở giao diện mạng, có lần không. Nguyên nhân gốc chưa được xác định. Giải pháp tạm thời là unit projectb-phy-retry thử gắn lại PHY tối đa 10 lần; trong 5 lần khởi động thử nghiệm, cả 5 lần board đều nhận địa chỉ IP mà không cần can thiệp.

2.4. BOOTLOADER, EXTLINUX VÀ CHUỖI KHỞI ĐỘNG

Board có sẵn U-Boot 2019.04 trên eMMC. Khi không giữ nút S2, ROM chạy bản trên eMMC, nghĩa là cấu hình U-Boot build trong image không có hiệu lực. Sau khi sao lưu có kiểm tra checksum, SPL trên eMMC được vô hiệu bằng cách ghi 0 vào sector 256; từ đó mọi lần khởi động đều chạy SPL và U-Boot 2025.01 trên thẻ SD. Chuỗi khởi động của image nộp ở Hình 2.2.



Hình 2.2. Chuỗi khởi động của image nộp

extlinux.conf không để wic tự sinh mà lấy từ layer (bootloader --configfile), để cmdline và dòng nạp overlay được quản lý trong mã nguồn chứ không sửa tay trên thẻ:

```
label Yocto
   kernel /zImage
   fdtdir /
   fdtoverlays /overlays/mpu6050.dtbo
   append root=/dev/mmcblk0p2 rootwait console=ttyS0,115200
```

2.5. KERNEL: LINUX-YOCTO, LINUX-YOCTO-RT VÀ KERNEL DEBUG

Phần cấu hình chung cho cả hai kernel đặt trong projectb-kernel.inc, được hai tệp .bbappend (linux-yocto và linux-yocto-rt) cùng require: biên dịch dtb với -@ để ghép được overlay, bật hid-multitouch cho màn cảm ứng. Kernel RT được chọn bằng PREFERRED_PROVIDER_virtual/kernel; linux-yocto-rt trong Poky chỉ khai cho máy ảo nên bbappend thêm COMPATIBLE_MACHINE và KMACHINE để dùng cấu hình beaglebone-preempt-rt có sẵn trong kernel-cache.

Để phát triển driver, một fragment bật PROVE_LOCKING (lockdep) và DEBUG_ATOMIC_SLEEP, điều khiển bằng biến PROJECTB_KERNEL_DEBUG. Fragment này đổi kích thước spinlock_t nhưng không đổi chuỗi vermagic, nên module build cho kernel thường vẫn nạp được vào kernel debug và hỏng âm thầm. Cách chặn: thêm hậu tố -debug vào chuỗi phiên bản; phép thử âm nạp module RT thường vào kernel RT debug bị từ chối với thông báo version magic. Image nộp dùng kernel RT không debug (6.12.47-rt6-yocto-preempt-rt).

Cấu hình của kernel RT và kernel thường trong kernel-cache khác nhau khoảng 140 dòng ngoài phần preemption (-O2 so với -Os, KFENCE, BPF...). Đề tài ghi nhận điểm này; việc san bằng cấu hình để so độ trễ công bằng thuộc giai đoạn 2.

2.6. DEVICE TREE OVERLAY CHO MPU6050

Overlay khai node imu@68 dưới &i2c2, ngắt từ GPIO1 chân 28 kiểu cạnh lên, cấu hình chân P9_12 ở mode 7 (GPIO) đầu vào kéo xuống, và đặt tốc độ I2C2 lên 400 kHz (lý do ở mục 3.3.3). Chân SDA, SCL không khai lại vì device tree gốc đã có.

```
&i2c2 {
    clock-frequency = <400000>;
    imu@68 {
        compatible = "projectb,mpu6050";
        reg = <0x68>;
        pinctrl-names = "default";
        pinctrl-0 = <&mpu6050_int_pins>;
        interrupt-parent = <&gpio1>;
        interrupts = <28 IRQ_TYPE_EDGE_RISING>;
    };
};
```

Lần build đầu, U-Boot chỉ in "Failed to apply overlay" rồi khởi động tiếp, overlay bị bỏ qua: dtb gốc chưa có __symbols__. Thêm KERNEL_DTC_FLAGS = "-@" thì U-

Boot ghép được, /proc/device-tree có node imu@68 và I2C core tạo client 2-0068 với modalias of:NimuT(null)Cprojectb,mpu6050. Thanh ghi pad của P9_12 (0x44E10878) đọc bằng devmem chuyển từ 0x37 sang 0x27 khi driver bind: pinctrl của thiết bị chỉ được áp lúc probe.

2.7. GHI THẺ VÀ KIỂM CHỨNG BRING-UP

Đầu đọc thẻ SD qua USB 3.0 từng ghi hỏng dữ liệu mà không báo lỗi khi ghi bằng một lệnh dd duy nhất (tốc độ báo về ở mức hàng GB/s). Script flash-verified.sh ghi từng khối 1 MB rồi đọc lại so SHA256, ghi lại khối sai; mọi lần ghi thẻ của đề tài đều dùng script này.

Bring-up cảm biến được kiểm theo thứ tự từ dưới lên: i2cdetect thấy địa chỉ 0x68; i2cget đọc WHO_AM_I = 0x70; logic analyzer ghi lại đúng giao dịch đó (Hình 2.3): START, địa chỉ ghi 0x68, thanh ghi 0x75, repeated START, địa chỉ đọc, byte 0x70 với NACK của master, STOP. Ở 100 kHz, chu kỳ SCL đo được 10.17 µs. Dữ liệu thô khi đặt phẳng và lật úp cho độ nhạy trục Z 16402 LSB/g (lệch 0.1% so với 16384 danh định).



Hình 2.3. Logic analyzer: đọc thanh ghi WHO_AM_I, chip trả 0x70

Kernel RT khởi động được với overlay, /sys/kernel/realtime bằng 1, chịu tải stress-ng 120 giây không cảnh báo. Mốc này được gắn tag bringup-done trước khi bắt đầu viết driver.

2.8. QUY TRÌNH PHÁT TRIỂN DRIVER

Ghi thẻ cho mỗi lần sửa driver mất vài phút và cần thao tác tay, nên driver được phát triển theo một vòng ngắn hơn: build riêng recipe module, chép tệp .ko lên board qua SSH (so SHA256 hai đầu), gỡ bản cũ và nạp bản mới, chạy bộ kiểm thử từ máy chủ. Một vòng như vậy mất khoảng một phút. Driver chỉ vào image ở tuần 8, khi giao diện và hành vi đã ổn định.

Suốt tuần 4–8, driver chạy trên kernel debug có lockdep: mọi đường khóa mới đều được kiểm ngay lần đầu chạy qua, và tiêu chí của mỗi bước gồm "lockdep vẫn còn hoạt động" (debug_locks = 1). Số đo thời gian chỉ được tin trên kernel không debug, vì lockdep làm chậm cả đường ngắt; mục 3.3.4 là ví dụ về một hằng số đo trên kernel debug phải sửa khi chuyển sang kernel nộp.

Với mỗi bước, tiêu chí kiểm tra (kết quả mong đợi và lý do) được viết ra trước khi thực hiện. Khi kết quả khác kỳ vọng, từng điều kiện (tốc độ lấy mẫu, cỡ lô, tốc độ

bus, kernel) được thay đổi lần lượt để tách riêng từng giả thuyết. Các thay đổi ảnh hưởng tới thời gian được so sánh với phiên bản trước trong cùng một lần khởi động, nhằm loại trừ sai khác do nhiệt độ, tải nền hoặc trạng thái bộ nhớ đệm. Kết quả, số đo và quyết định của từng tuần được ghi vào nhật ký tiến độ trong repo.

CHƯƠNG 3. THIẾT KẾ, HIỆN THỰC VÀ KIỂM THỬ DRIVER MPU6050

3.1. YÊU CẦU VÀ HỢP ĐỒNG GIAO DIỆN

3.1.1. Yêu cầu

- Ứng dụng nhận mẫu liên tục, mỗi mẫu có timestamp chính xác tới cỡ chục micro giây, không mất mẫu ở tốc độ cam kết, kể cả khi CPU chịu tải.
- Ứng dụng dùng poll() để chờ dữ liệu, không hỏi vòng; cấu hình đổi được khi đang đọc.
- Mẫu bị mất phải được đếm, không được mất âm thầm.
- Lỗi phần cứng (bus I2C, chip mất nguồn) không làm treo nhân hay treo ứng dụng; hết lỗi thì tự phục hồi.
- Gỡ driver khi ứng dụng còn mở thiết bị không được gây lỗi bộ nhớ.

Giao diện được chốt ở tuần 5 và giữ nguyên tới giai đoạn 2: dữ liệu chỉ đi qua /dev/mpu6050, sysfs chỉ dùng cho cấu hình. Đọc dữ liệu qua sysfs bị loại vì mỗi lần đọc chỉ được một giá trị dạng văn bản, không có timestamp, không poll được theo từng mẫu, và tốn một lần chuyển đổi số sang chuỗi rồi ngược lại.

3.1.2. Header uapi

Header include/uapi/projectb/mpu6050.h dùng chung cho driver và ứng dụng (Bảng 3.1). Bản ghi dài 16 byte trên mọi ABI: bốn số 16 bit đứng trước nên ts_ns luôn ở offset 8, không phụ thuộc kiến trúc căn số 64 bit theo 4 hay 8 byte. Kích thước các struct được kiểm bằng static_assert lúc biên dịch; thử cố ý đặt sai thì assert bắt được trên i386.

Bảng 3.1. Hợp đồng /dev/mpu6050

Phần tử	Mô tả
struct mpu6050_sample (16 B)	s16 ax, ay, az, temp (giá trị thô); u64 ts_ns (CLOCK_MONOTONIC)
read()	trả bội số 16 byte, chỉ bản ghi trọn; rỗng thì chặn hoặc EAGAIN
poll()	POLLIN khi có bản ghi; POLLHUP | POLLERR khi thiết bị bị gỡ
MPU6050_IOC_GET_CONFIG	tốc độ thật, dải đo, LSB/g, băng thông lọc, hệ số nhiệt độ
MPU6050_IOC_GET_STATS	dropped: số mẫu mất từ lúc probe
MPU6050_IOC_FLUSH	bỏ các bản ghi đang chờ đọc
open()	một người mở; lần mở thứ hai EBUSY; mở là bật nguồn dữ liệu

3.1.3. Thuộc tính sysfs

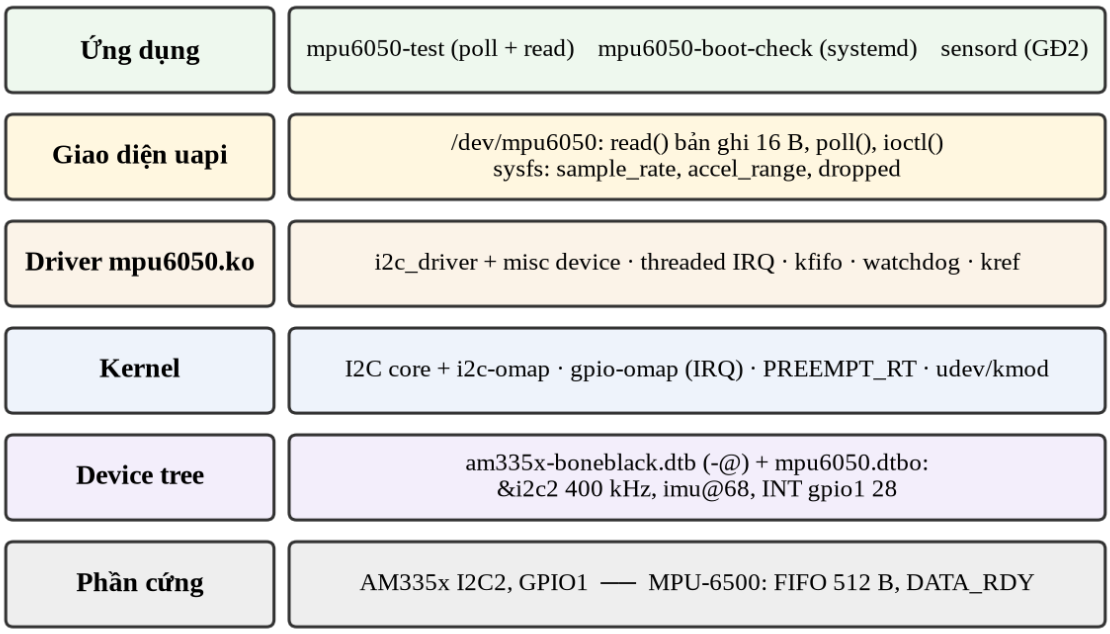
Bảng 3.2. Thuộc tính sysfs trong /sys/class/misc/mpu6050/

Thuộc tính	Mô tả
sample_rate	ghi 4–1000 Hz; đọc lại tốc độ thật (ghi 300, đọc 333 vì SMPLRT_DIV nguyên)
accel_range	ghi 2, 4, 8, 16 (g)
dropped	chỉ đọc; số mẫu chip đã tạo ra (có cạnh INT) mà driver làm mất

Đổi tốc độ thì đổi luôn bộ lọc thông thấp: driver chọn băng thông lớn nhất trong {184, 92, 41, 20, 10, 5} Hz thỏa 2 × băng thông ≤ tốc độ lấy mẫu (điều kiện Nyquist). Đổi cấu hình khi đang đọc thì dừng nguồn dữ liệu, ghi thanh ghi, xóa fifo, chạy lại, nên mọi bản ghi đọc được sau khi lệnh ghi sysfs trả về đều theo cấu hình mới.

3.2. KIẾN TRÚC DRIVER

Các tầng của hệ thống và giao diện giữa chúng ở Hình 3.1. Driver không biết chân nào, bus nào: địa chỉ, ngắt, kiểu cạnh lấy từ device tree, tốc độ bus lấy từ clock-frequency của bộ điều khiển. Ứng dụng không biết thanh ghi: chỉ thấy bản ghi và các hệ số chuyển đổi qua ioctl, nên đổi chip 6050/6500 không phải sửa ứng dụng.

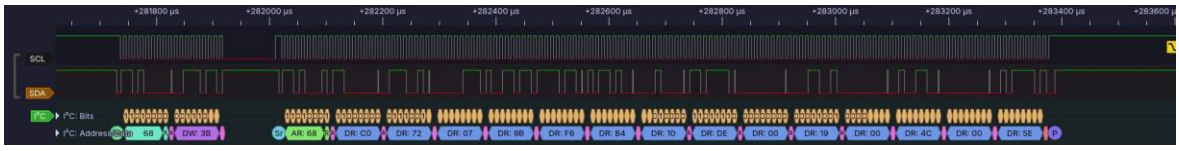


Hình 3.1. Các tầng của hệ thống và giao diện giữa các tầng

3.2.1. Probe, reset và cấu hình chip

Probe đọc WHO_AM_I để chọn bảng thông số (6500 hoặc 6050), kiểm device tree có ngắt cạnh lên, rồi reset chip bằng DEVICE_RESET. Không dựa vào giá trị lúc cấp nguồn vì insmod/rmmod không làm chip mất nguồn. Sau reset, driver ghi tường minh mọi thanh ghi cấu hình và đọc lại để kiểm; ghi mà đọc lại khác thì probe thất bại. Nguồn nhịp chọn PLL theo con quay (chính xác hơn bộ dao động trong), nên con quay vẫn bật dù driver không xuất dữ liệu con quay. Cuối cùng mới đăng ký misc device, để ứng dụng chỉ thấy /dev/mpu6050 khi mọi thứ đã sẵn sàng.

Dữ liệu luôn được đọc bằng một giao dịch burst. Hình 3.2 là lần đọc 14 byte từ 0x3B trong lúc probe của bản driver tuần 4 (bản cuối đọc qua FIFO, cũng theo burst): 1.66 ms ở 100 kHz, trong khi đọc lẻ 14 thanh ghi mất khoảng 6.6 ms và có thể xé giá trị (byte cao của mẫu này, byte thấp của mẫu sau). Byte giải mã từ dây trùng từng số với dòng dmesg driver in ra trong cùng lần probe.



Hình 3.2. Logic analyzer: một giao dịch burst 14 byte từ thanh ghi 0x3B

3.2.2. Char device

open đặt cờ mở bằng test_and_set_bit_lock (lần mở thứ hai nhận EBUSY), xóa fifo và bật nguồn dữ liệu; release tắt nguồn rồi mới nhả cờ. read chỉ trả bản ghi trọn; buffer nhỏ hơn 16 byte nhận EINVAL. Khi fifo rỗng, read ngủ trên wait queue mà không

giữ khóa, vì đổi cấu hình cần lấy khóa phía đọc để xóa fifo; giữ khóa trong lúc ngủ sẽ thành chờ nhau mãi. fops.owner = THIS_MODULE khiến rmmod bị chặn khi còn tệp mở. Driver không khai .write và .llseek: write trên tệp mở chỉ đọc bị VFS chặn trước (EBADF), còn stream_open làm lseek trả ESPIPE.

```
static const struct file_operations mpu6050_fops = {
    .owner         = THIS_MODULE,   /* chặn rmmod khi còn tệp mở */
    .open          = mpu6050_open,
    .release       = mpu6050_release,
    .read          = mpu6050_read,
    .poll          = mpu6050_poll,
    .unlocked_ioctl = mpu6050_ioctl,
    .compat_ioctl   = compat_ptr_ioctl,
};
```

3.2.3. Nguồn dữ liệu qua các bước

Phía đọc (kfifo, read, poll) và nguồn dữ liệu được tách nhau, nên nguồn dữ liệu thay đổi qua ba bước mà ứng dụng không phải sửa:

- Tuần 5: hrtimer + work đọc từng mẫu. delayed_work không cho ra đúng 10 ms vì HZ = 100 và timer luôn cộng thêm một tick để không chạy sớm. Nhịp hrtimer độc lập với đồng hồ chip nên thỉnh thoảng đọc lại mẫu cũ (31 bản ghi trùng trong lượt đo ở 333 Hz).
- Tuần 6: ngắt DATA_RDY, đọc từng mẫu trong thread ngắt. Hết bản ghi trùng, 30 phút tải nền không mất mẫu ở 100 Hz, nhưng 1000 Hz không đạt: mỗi mẫu một giao dịch 1.1 ms ở 100 kHz.
- Tuần 7: chip ghi vào FIFO, thread đọc theo lô; mỗi mẫu mang timestamp của chính cạnh INT của nó; I2C 400 kHz.

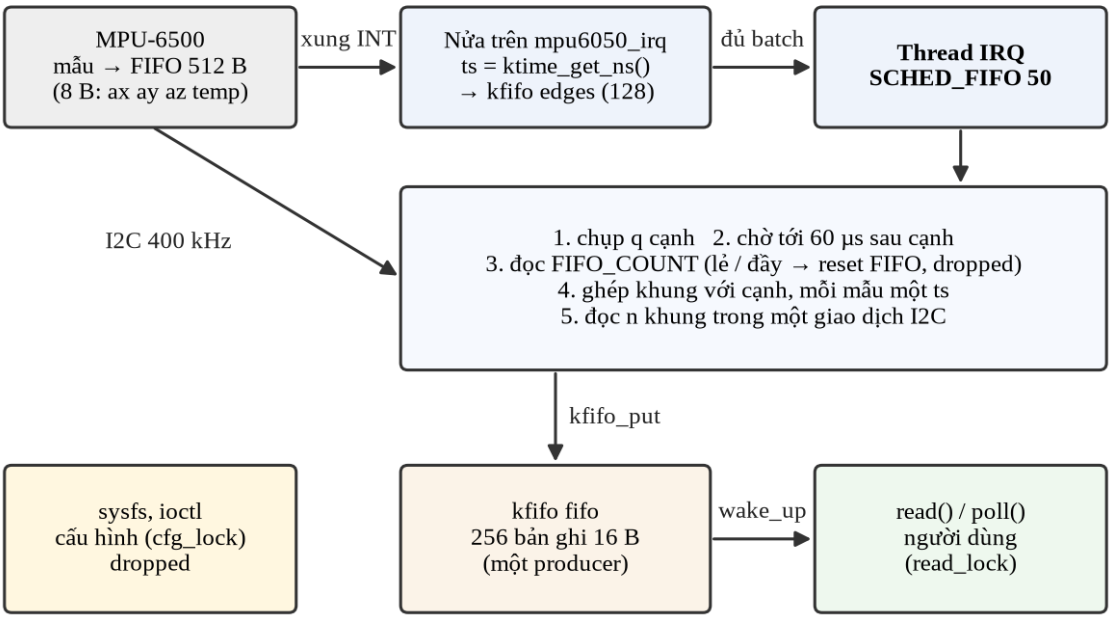
Bảng 3.3 so sánh ba cách lấy dữ liệu bằng cùng các phép đo. Mỗi bước giải quyết đúng giới hạn mà bước trước để lộ ra: đồng hồ của nhân không khớp đồng hồ chip thì chuyển sang ngắt của chip; mỗi mẫu một giao dịch I2C thì không đủ thời gian ở tốc độ cao, chuyển sang FIFO theo lô.

Bảng 3.3. So sánh ba cách lấy dữ liệu

Phép đo	hrtimer + work	Ngắt, từng mẫu	FIFO theo lô
Bản ghi trùng	31 (333 Hz)	0 / 6000 (333 Hz)	0 / 3000 (100 Hz)
Tải stress-ng, 100 Hz	83 bản ghi/s	101–102/s, dropped 0	101–102/s, dropped 0
1000 Hz	chưa đo	103 bản ghi/s, ~800 mất/s	bản ghi = số ngắt, dropped 0 (kernel nộp)
Timestamp	lúc work chạy	cạnh INT	cạnh INT của từng mẫu

3.3. NGẮT DATA_RDY VÀ ĐỌC FIFO THEO LÔ

Luồng dữ liệu trong driver ở Hình 3.3. Chip không có ngắt "FIFO đủ N mẫu", nên driver vẫn nhận ngắt mỗi mẫu nhưng chỉ đánh thức thread khi đủ một lô.



Hình 3.3. Luồng dữ liệu trong driver

3.3.1. Nửa trên và hàng đợi cạnh

Nửa trên chỉ làm hai việc: ghi thời điểm cạnh (ktime_get_ns) vào kfifo edges 128 phần tử, và đánh thức thread khi số cạnh mới đủ batch. gpio-omap xử lý ngắt cạnh bằng handle_simple_irq, không che ngắt, nên cạnh đến trong lúc thread đang đọc vẫn vào nửa trên: mỗi mẫu có đúng một timestamp. Một lỗi đã gặp: so tổng số cạnh trong hàng đợi với batch thì các cạnh thread đang xử lý cũng bị đếm, thread vừa xong lô đã chạy lại khi cạnh mới nhất đã cũ, 977/1011 lô ở 1000 Hz không ghép được timestamp. Sửa bằng cách trừ số cạnh đang xử lý (inflight). Toàn bộ nửa trên:

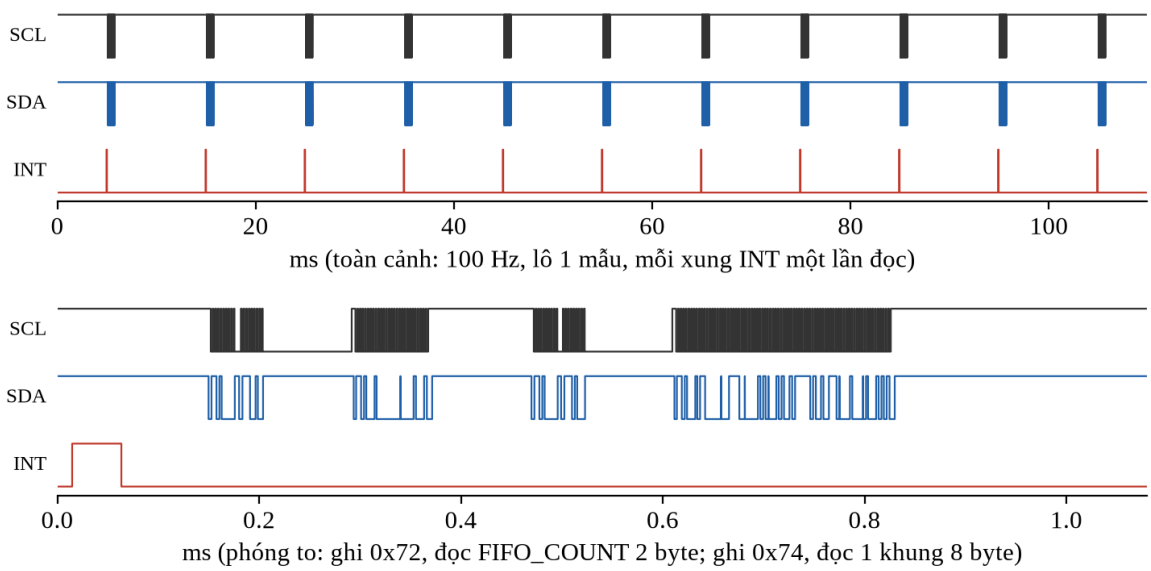
```
static irqreturn_t mpu6050_irq(int irq, void *data)
{
    struct mpu6050 *st = data;

    WRITE_ONCE(st->n_edges, st->n_edges + 1);      /* cho watchdog */
    if (!kfifo_put(&st->edges, ktime_get_ns())) {  /* hàng đợi cạnh đầy
*/
        atomic64_inc(&st->dropped);
        atomic_inc(&st->edges_lost);
    }
    return kfifo_len(&st->edges) - READ_ONCE(st->inflight) >= st-
>batch ?
        IRQ_WAKE_THREAD : IRQ_HANDLED;
}
```

3.3.2. Thread ngắt: ghép khung với cạnh

Thread chụp q cạnh, đọc FIFO_COUNT (n khung), rồi ghép khung với cạnh. Khó khăn là biết khung mới nhất trong FIFO thuộc cạnh nào: có thể một mẫu vào FIFO sau khi chụp cạnh nhưng trước khi count được chốt. Driver chỉ "neo" khung mới nhất vào cạnh mới nhất khi chắc chắn không mẫu nào lọt vào khoảng đó (thời gian từ cạnh mới nhất tới lúc đọc xong count, cộng lề, nhỏ hơn một chu kỳ). Neo được thì n < q nghĩa là q − n mẫu cũ nhất đã mất (cộng dropped), n > q nghĩa là có mẫu không có cạnh: sau khi bật là mẫu vào FIFO trước khi ngắt mở, timestamp lùi đúng một chu kỳ; đang chạy là cạnh bị dồn, mẫu được đặt vào khoảng giữa hai cạnh còn dư thời gian nhiều nhất.

Hình 3.4 ghi lại bus ở 100 Hz, 400 kHz. Ở tốc độ này một lô là một mẫu (10 ms dữ liệu), nên sau mỗi xung INT có một lần đọc FIFO_COUNT (ghi 0x72, repeated START, đọc 2 byte) và một lần đọc FIFO_R_W (ghi 0x74, đọc 8 byte). Cả 200 lần đọc count đều ra 8 byte; timestamp lệch so với cạnh INT trong khoảng −59..+52 µs quanh trung vị. Ở bản ghi tương tự với I2C 100 kHz, đối chiếu từng khung trên dây với bản ghi ứng dụng nhận được: 500/500 khung trùng.



Hình 3.4. Logic analyzer: INT và đọc FIFO ở 100 Hz, I2C 400 kHz

3.3.3. Cỡ lô theo tốc độ bus và hiện tượng trượt con trỏ FIFO

Ở 1000 Hz, dữ liệu xuất hiện các mẫu có nhiệt độ −77..+117 °C và khung toàn 0xFF. Logic analyzer cho thấy FIFO_COUNT không còn chia hết cho 8 và dữ liệu lệch 2 byte. Tách điều kiện từng bước (bộ lọc, cỡ lô, tốc độ) cho thấy hiện tượng xảy ra khi lần đọc lô kéo dài qua thời điểm chip ghi mẫu kế: khoảng 1% số lần như vậy, con trỏ FIFO của chip trượt vài byte và mọi khung sau đó lệch. Datasheet không nhắc tới hiện tượng này.

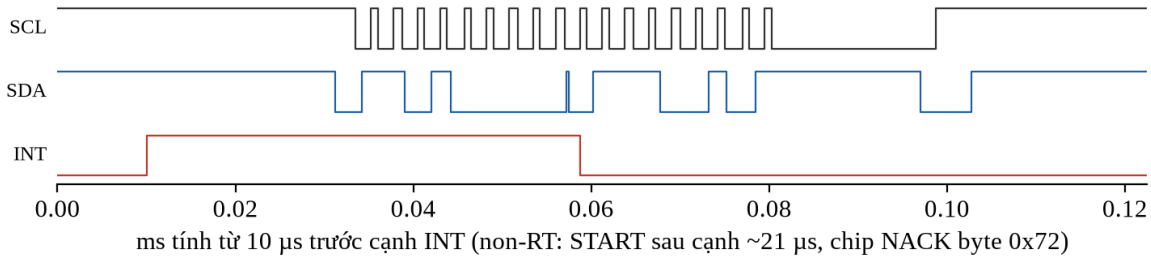
Driver xử lý bằng ba biện pháp: chọn cỡ lô sao cho đọc xong trước mẫu kế; phát hiện count không chia hết cho 8 thì reset FIFO và tính cả lô vào dropped; reset FIFO theo trình tự của driver mainline (tắt ghi trước khi reset). Cỡ lô bằng số mẫu trong khoảng 10 ms, nhưng không vượt số khung đọc kịp: một khung trên dây là 8 byte × 9 bit = 72 nhịp (180 µs ở 400 kHz), ngân sách là chu kỳ trừ 450 µs (từ cạnh tới lúc bắt đầu đọc khung, đo trên LA) trừ 150 µs lề. Kết quả ở Bảng 3.4; ở 1000 Hz, giới hạn đọc kịp quyết định cỡ lô. Đây cũng là lý do chuyển I2C lên 400 kHz: ở 100 kHz, riêng đọc count mất trên 0.45 ms và 1000 Hz không còn chỗ cho lô nào.

Bảng 3.4. Cỡ lô theo tốc độ lấy mẫu ở I2C 400 kHz

Tốc độ	Chu kỳ	Đọc kịp (khung)	Cỡ lô	Lô mỗi giây
100 Hz	10 ms	52	1	100
200 Hz	5 ms	24	2	100
333 Hz	3 ms	13	4	83
500 Hz	2 ms	7	5	100
1000 Hz	1 ms	2	2	500

3.3.4. Chờ sau cạnh INT

Khi chạy trên kernel thường, thread được đánh thức rất sớm và đọc count chỉ 21 µs sau cạnh INT, trong lúc xung INT 50 µs chưa hết; chip trả NACK cho byte địa chỉ thanh ghi (Hình 3.5). Chờ tới 60 µs sau cạnh thì không còn NACK. Driver chỉ có timestamp (lấy ở nửa trên), không có thời điểm cạnh thật, nên chờ tới timestamp + 60 µs trừ đi độ trễ tối thiểu của timestamp so với cạnh.



Hình 3.5. Logic analyzer: giao dịch bắt đầu trong xung INT bị chip NACK

Độ trễ đó phụ thuộc cấu hình kernel. Đo bằng ftrace (irq 26 của bank GPIO gần như trùng cạnh, irq 50 là nơi lấy timestamp), trên kernel RT debug độ trễ ít nhất 40 µs do lockdep làm chậm thread irq/26, còn trên kernel RT không debug chỉ 6–18 µs. Bản đầu trừ 40 µs cho mọi kernel RT, nên trên image nộp vẫn đọc ở 31–37 µs sau cạnh và bị NACK 1.3% ở 500 Hz, 8.6% ở 1000 Hz. Sửa: chỉ trừ khi kernel vừa RT vừa có lockdep. So sánh trong cùng lần khởi động ở Bảng 3.5.

Bảng 3.5. Đọc FIFO_COUNT trên kernel nộp trước và sau khi sửa (ftrace, 30000 bản ghi)

	Trước khi sửa	**Sau khi sửa**
Đọc count sau cạnh (500 Hz)	31–55 µs	70–81 µs
NACK ở 500 Hz	79 / 6063	0 / 6000
NACK ở 1000 Hz	1349 / 15679	0 / 15000
FIFO lệch, dropped ở 1000 Hz	4 lần, 8 mẫu	0, 0

Từ trường hợp này, mọi hằng số thời gian xác định trên kernel debug đều cần được đo lại trên kernel nộp. Đáng chú ý là chương trình tự kiểm lúc khởi động vẫn báo ĐẠT với bản lỗi, vì NACK chỉ khiến lô đó được đọc lại ở lần sau; lỗi chỉ thể hiện trong dmesg.

3.3.5. Tràn FIFO

Khi thread bị chặn quá lâu, FIFO đầy (512 byte, 64 mẫu, 640 ms ở 100 Hz). Driver phát hiện bằng count = 512, reset FIFO và cộng mọi cạnh đang chờ vào dropped. Thí nghiệm chặn thread 1 giây bằng tham số thử stall_ms cho khoảng trống timestamp 107 chu kỳ và dropped tăng đúng 106 (một mẫu dư sau reset). Nhận định "512 chia hết cho 8 nên tràn FIFO không làm lệch khung" đã được kiểm chứng bằng thực nghiệm: trong 4/4 lần cho FIFO tràn mà không reset, nhiệt độ và |a| vẫn đúng.

3.4. ĐỒNG BỘ VÀ KHÓA

Bảng 3.6 tóm tắt dữ liệu chia sẻ và cách bảo vệ. Thứ tự khóa là cfg_lock rồi read_lock, không bao giờ ngược lại. Thread ngắt không lấy khóa nào của driver, vì hàm dừng nguồn dữ liệu giữ cfg_lock và gọi disable_irq, hàm này chờ thread chạy xong; nếu thread cũng lấy cfg_lock thì hai bên chờ nhau. Lockdep không thấy kiểu chờ này, nên quy tắc được ghi thành chú thích trong mã. Lockdep không báo lỗi nào trong suốt quá trình phát triển trên kernel debug.

Bảng 3.6. Dữ liệu chia sẻ trong driver

Dữ liệu	Ghi	Đọc	Bảo vệ
kfifo edges (timestamp cạnh)	nửa trên	thread	một ghi một đọc, không khóa
kfifo fifo (bản ghi)	thread	read()	một producer; read_lock phía đọc
thanh ghi chip, cấu hình	open, release, sysfs, watchdog	—	cfg_lock (mutex)
dropped	nửa trên, thread	sysfs, ioctl	atomic64 (u64 trên ARM32)

3.5. ĐƯỜNG LỖI VÀ VÒNG ĐỜI

3.5.1. Gỡ thiết bị khi ứng dụng còn mở

Ban đầu trạng thái driver cấp bằng devm_kzalloc. Ghi unbind vào sysfs gọi remove ngay dù ứng dụng còn mở /dev/mpu6050, devres giải phóng vùng nhớ, và tệp đang mở trỏ vào vùng đã giải phóng. Sửa bằng kref: thiết bị giữ một tham chiếu, trả bằng devm action đăng ký đầu tiên (chạy cuối cùng, sau khi ngắt đã gỡ), mỗi tệp mở giữ một tham chiếu. remove đặt cờ gone và đánh thức người đang ngủ; từ đó read trả nốt bản ghi còn trong fifo rồi ENODEV, poll báo POLLHUP, ioctl trả ENODEV. Lấy tham chiếu trong open là an toàn vì misc_open gọi open khi giữ misc_mtx, và misc_deregister cũng lấy khóa này.

```
static void mpu6050_free(struct kref *ref)
{
    struct mpu6050 *st = container_of(ref, struct mpu6050, ref);

    mutex_destroy(&st->read_lock);
    mutex_destroy(&st->cfg_lock);
    kfree(st);
}

/* probe: action đăng ký đầu tiên nên chạy sau cùng, sau khi ngắt đã
gỡ */
kref_init(&st->ref);
ret = devm_add_action_or_reset(dev, mpu6050_put, st);
/* open: kref_get(&st->ref);   release: kref_put(&st->ref,
mpu6050_free); */
```

3.5.2. Lỗi bus I2C và chip mất nguồn

Hai loại sự cố này có biểu hiện khác nhau. Khi rút SDA, chip vẫn chạy và cạnh INT vẫn tới: thread đọc count lỗi thì giữ nguyên cạnh, hàng đợi cạnh đầy thì nửa trên tính mỗi cạnh mới vào dropped, cắm lại thì thấy FIFO đầy, reset và chạy tiếp. Trường hợp này không cần bổ sung mã xử lý riêng. Khi chip mất nguồn, không còn cạnh nào;

có nguồn lại, chip về giá trị reset (INT_ENABLE = 0) nên không tự chạy lại. Driver thêm một watchdog (delayed work 1 giây, chỉ chạy khi có tệp mở): một nhịp không có cạnh nào thì khởi tạo lại chip như lúc probe. Watchdog đọc WHO_AM_I trước, đọc lỗi thì thôi, không ghi gì và không in log; chỉ báo lần lỗi đầu và lúc hồi phục.

```
static void mpu6050_watchdog(struct work_struct *work)
{
    ...
    mutex_lock(&st->cfg_lock);
    if (!st->active) {               /* không còn tệp mở: không hẹn lại
*/
        mutex_unlock(&st->cfg_lock);
        return;
    }
    if (!st->streaming || edges == st->wd_edges) {
        ret = mpu6050_recover(st);     /* WHO_AM_I, reset, cấu hình,
start */
        ...
    }
    st->wd_edges = edges;
    schedule_delayed_work(&st->watchdog, msecs_to_jiffies(1000));
    mutex_unlock(&st->cfg_lock);
}
```

Watchdog lấy cfg_lock, nên release và remove phải gọi cancel_delayed_work_sync sau khi đã nhả cfg_lock; gọi trong lúc giữ khóa thì release chờ work, work chờ khóa. Cờ active được xóa trong khóa trước khi hủy, để work đang chạy dở không tự hẹn lại.

Trường hợp chip mất nguồn, dropped không tăng vì chip không tạo ra mẫu nào; khoảng mất thể hiện qua khoảng trống timestamp. Ghi sysfs lúc bus hỏng trả EREMOTEIO sau khoảng 0.15 giây, giữ cấu hình cũ; watchdog bật lại nguồn dữ liệu khi bus về. Log lỗi dùng dev_err_ratelimited, tối đa 10 dòng mỗi 5 giây.

3.6. TÍCH HỢP VÀO IMAGE

Chuỗi tự động khi khởi động: overlay tạo thiết bị 2-0068, uevent mang modalias, udev tra modules.alias (sinh từ MODULE_DEVICE_TABLE lúc dựng rootfs) và nạp mpu6050.ko, probe tạo /dev/mpu6050. Rule udev 90-mpu6050.rules ghi quyền tường minh (0600, root) và gắn thẻ systemd, kéo unit mpu6050-boot-check lên khi thiết bị xuất hiện. Unit chạy selftest và đo 1000 bản ghi ở 100 Hz; kết quả đọc bằng journalctl. Unit không có mục [Install] nên không làm chậm multi-user.target, và tự chạy lại sau mỗi lần unbind/bind.

```
KERNEL=="mpu6050", SUBSYSTEM=="misc", OWNER="root", GROUP="root",
MODE="0600", \
    TAG+="systemd", ENV{SYSTEMD_WANTS}+="mpu6050-boot-check.service"
```

3.7. KIỂM THỬ VÀ ĐÁNH GIÁ

3.7.1. Phương pháp và công cụ

Mỗi bước có tiêu chí kiểm tra cụ thể trên board; mọi lệnh dài đặt trong script để chạy lại được (Bảng 3.7). Mỗi phép đo kiểm cả số lượng và nội dung: số ngắt phải bằng số bản ghi cộng dropped; gia tốc khi đứng yên phải gần 1 g; nhiệt độ phải hợp lý (một khung lệch byte cho ra nhiệt độ vô lý). Thay đổi nào ảnh hưởng thời gian thì so sánh với bản trước trong cùng lần khởi động.

Bảng 3.7. Bộ kiểm thử

Script / chương trình	Kiểm tra
mpu6050-test -t	selftest 13 mục: EAGAIN, EBUSY, EINVAL, ENOTTY, ESPIPE, poll, ioctl, ts đơn điệu
mpu-check.sh sweep, one, cfg	tốc độ 100–1000 Hz, bảo toàn, đổi cấu hình khi đang đọc
mpu-check.sh oflow	tràn FIFO cố ý, có và không reset
mpu-stress.sh load, oc, sysfs, mod	tải stress-ng, mở đồng thời, ghi sysfs ngẫu nhiên, nạp/gỡ
mpu-fault.sh watch, unbind, held	rút dây, mất nguồn, gỡ thiết bị khi đang đọc
mpu-trace.sh, mpu-lag.sh	ftrace: cạnh → timestamp → giao dịch I2C, NACK
la-fifo.py, mpu-ts.py	ghép khung trên dây với bản ghi; phân tích khoảng timestamp

3.7.2. Kết quả trên kernel nộp

Kết quả chạy trên image nộp (kernel 6.12.47-rt6-yocto-preempt-rt không debug) ở Bảng 3.8. Tốc độ cam kết cho mốc không mất mẫu là 500 Hz, đạt trên mọi kernel đã thử; 1000 Hz sạch trên kernel nộp và kernel thường, còn khoảng 0.03% mất trên kernel debug.

Bảng 3.8. Kết quả kiểm thử trên kernel nộp

Phép thử	Kết quả
selftest	0 FAIL
sweep 100–1000 Hz, 10 s mỗi mức	dropped 0, FIFO lệch 0, CPU rảnh 90–98%
500 và 1000 Hz, 30000 bản ghi, ftrace	NACK 0, đọc count 70–81 µs sau cạnh
load 600: stress-ng cpu + io + vm 10 phút, 100 Hz	60568 bản ghi = số ngắt, dropped 0, khoảng ts lớn nhất 10.2 ms
tràn cố ý 1 s	dropped +107, dư 0
đổi cấu hình khi đọc	100 → 200 Hz, ±2 → ±4 g, không bản ghi lẫn hai cấu hình
tự kiểm lúc khởi động	ĐẠT, 19 s sau khởi động, không thao tác tay

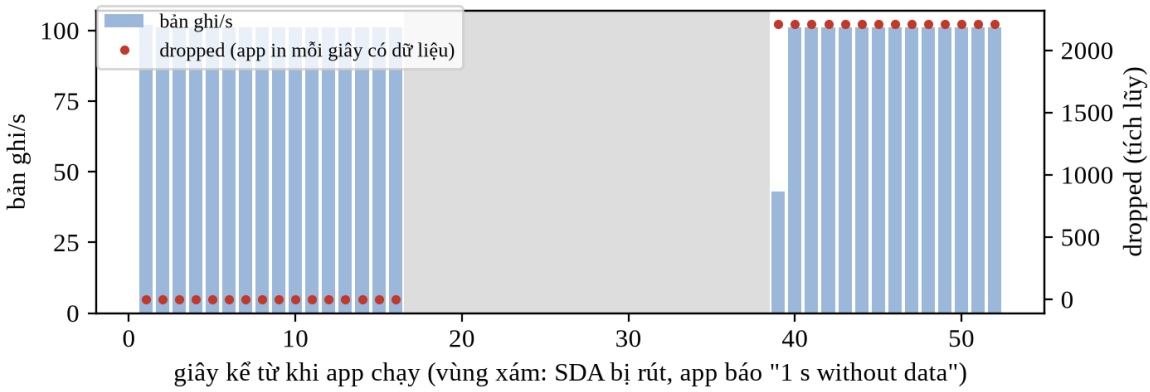
So với các bước trước: tuần 5 (hrtimer + work) dưới tải stress-ng chỉ còn 83 bản ghi/giây ở 100 Hz; tuần 6 (ngắt, đọc từng mẫu) đạt 100 Hz nhưng 1000 Hz chỉ còn 103 bản ghi/giây; tuần 7–8 (FIFO theo lô) đạt 1000 Hz không mất mẫu trên kernel nộp. Khoảng timestamp lớn nhất dưới tải là 10.2 ms trên kernel nộp (10 phút); lượt 30 phút ở tuần 7 cho 26.2 ms trên kernel debug và 10.6 ms trên kernel thường.

3.7.3. Đường lỗi

Các phép thử đường lỗi chạy trên kernel debug để lockdep theo dõi, kết quả ở Bảng 3.9. Hình 3.6 là lần rút SDA trong buổi diễn tập demo: SDA bị rút 22.1 giây ở 100 Hz, dropped tăng 2210, ứng dụng báo "1 s without data" mỗi giây rồi tự chạy tiếp khi cắm lại.

Bảng 3.9. Kết quả kiểm thử đường lỗi

Phép thử	Kết quả
rút SDA 17.9 s, 100 Hz	dropped +1795 (17.9 s × 100 = 1792), tự hồi, không cần rmmod
rút SDA rồi ghi sysfs	EREMOTEIO sau 0.15 s, cấu hình giữ nguyên; watchdog bật lại khi bus về
chip mất nguồn 14.7 s	watchdog khởi tạo lại chip; 41641 bản ghi, 0 khung lệch byte
unbind khi ứng dụng đang đọc, 20 vòng	0 lỗi; vùng nhớ cấp 20 lần, giải phóng đủ; slab không tăng
unbind khi tệp mở nhưng không đọc	đọc được 110 bản ghi còn lại rồi ENODEV
log lỗi	tối đa 10 dòng/5 s, phần còn lại gộp "callbacks suppressed"

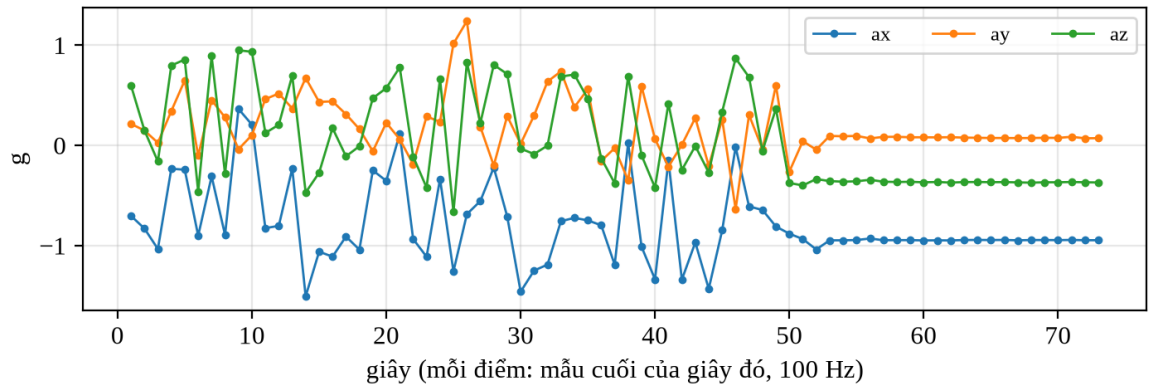


Hình 3.6. Rút SDA khi đang đọc ở 100 Hz: bản ghi mỗi giây và dropped

Việc giải phóng vùng nhớ khi unbind được kiểm bằng event kmem:kmalloc và kmem:kfree của ftrace, ghép theo địa chỉ con trỏ: 18 lần giải phóng từ devres (ứng dụng đóng tệp trước) và 1 lần từ release (tệp đóng sau), cho thấy cả hai đường của kref đều chạy.

3.7.4. Demo

Kịch bản demo gồm 6 bước, chạy bằng script demo.sh trên image nộp: kernel và overlay, kết quả tự kiểm lúc khởi động, đọc liên tục khi nghiêng module (Hình 3.7), đổi tốc độ 100 → 500 Hz và dải ±8 g khi đang đọc, rút SDA và tự phục hồi, gỡ driver khi ứng dụng đang đọc. Buổi diễn tập chạy đủ 6 bước, dropped bằng 0 ở mọi bước trừ bước rút dây.



Hình 3.7. Gia tốc ba trục khi nghiêng module trong buổi diễn tập demo

3.7.5. Hạn chế

- Trên kernel RT debug đã quan sát cạnh INT tới nửa trên trễ vài mili giây khi một kworker giữ khóa BH; không mất mẫu (FIFO giữ) nhưng timestamp của cạnh đó trễ, và ở 1000 Hz việc này đẩy lô sang lúc chip ghi, vài khung hỏng lọt ra trước khi bị phát hiện. Kernel nộp chưa được đo riêng hiện tượng này; khoảng timestamp lớn nhất dưới tải là 10.2 ms.

- Độ trễ timestamp so với cạnh là hằng số chọn theo cấu hình kernel; đổi kernel phải đo lại.
- dropped gộp mọi nguyên nhân mất mẫu; không tính mẫu khi chip mất nguồn.
- Ghi sysfs lỗi lúc không có tệp mở có thể để chip ở cấu hình dở dang tới lần ghi hay probe sau.
- Chỉ một chip, tên /dev/mpu6050 cố định; chưa xuất dữ liệu con quay.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết luận

Đề tài đã đạt được các mục tiêu đề ra. Về BSP, image Yocto được xây dựng từ một layer riêng với kernel PREEMPT_RT; cmdline, extlinux và overlay đều được quản lý trong mã nguồn. Khi build lại từ đầu (sstate trống), zImage và device tree thu được trùng SHA256 với bản trước. Về driver, module MPU6050 ngoài cây cung cấp /dev/mpu6050 với read, poll, ioctl và cấu hình qua sysfs, sử dụng ngắt DATA_RDY và FIFO của chip, mỗi mẫu mang timestamp của chính cạnh ngắt tương ứng. Trên kernel nộp, driver đạt 1000 Hz không mất mẫu, và ở 100 Hz không mất mẫu khi chịu tải stress-ng trong 10 phút. Driver xử lý được lỗi bus, chip mất nguồn, tràn FIFO và trường hợp gỡ thiết bị khi đang đọc; driver tự nạp và tự kiểm tra khi khởi động. Mã nguồn được quản lý bằng Git với tag v1.0-chuyennganh.

Kinh nghiệm quan trọng rút ra từ đề tài là phải đo đạc trên phần cứng thật thay vì chỉ dựa vào tài liệu. Hai hiện tượng ảnh hưởng lớn nhất tới driver (chip trượt con trỏ FIFO khi bị đọc lúc đang ghi, chip trả NACK trong thời gian phát xung ngắt) đều không được mô tả trong datasheet và chỉ được phát hiện nhờ logic analyzer và ftrace.

Hướng phát triển

- San bằng cấu hình kernel RT và kernel thường, đo độ trễ thời gian thực bằng cyclictest và độ trễ từ ngắt tới ứng dụng, có và không có tải.
- Middleware sensord (C++17): đọc /dev/mpu6050 bằng poll, phân phối dữ liệu qua shared memory và message queue, chạy không quyền root.
- Giao diện người dùng trên màn 7 inch và phát hiện bất thường đơn giản.
- Machine conf riêng, watchdog phần cứng, tách image sản phẩm và image phát triển; thiết kế cập nhật OTA A/B.
- Xuất dữ liệu con quay; cân nhắc bọc driver bằng giao diện IIO.

DANH MỤC TÀI LIỆU THAM KHẢO

Tiếng Anh:

Corbet, J., Rubini, A., Kroah-Hartman, G. (2005), Linux Device Drivers, 3rd edition, O'Reilly Media, Sebastopol.

InvenSense (2013), MPU-6000 and MPU-6050 Register Map and Descriptions, RM-MPU-6000A-00, revision 4.2.

InvenSense (2013), MPU-6500 Register Map and Descriptions, RM-MPU-6500A-00, revision 2.1.

InvenSense (2014), MPU-6500 Product Specification, PS-MPU-6500A-01, revision 1.1.

Madieu, J. (2017), Linux Device Drivers Development, Packt Publishing, Birmingham.

NXP Semiconductors (2021), I2C-bus specification and user manual, UM10204, revision 7.0.

Simmonds, C., Vasquez, F. (2021), Mastering Embedded Linux Programming, 3rd edition, Packt Publishing, Birmingham.

Texas Instruments (2019), AM335x and AMIC110 Sitara Processors Technical Reference Manual, SPRUH73Q.

Website:

BeagleBoard.org, "BeagleBone Black System Reference Manual", https://docs.beagleboard.org, truy cập ngày 15/10/2026.

Linux kernel, "The Linux Kernel documentation", https://docs.kernel.org, truy cập ngày 30/10/2026.

Linux kernel, "drivers/iio/imu/inv_mpu6050", https://git.kernel.org (nhánh v6.12), truy cập ngày 01/11/2026.

The Yocto Project, "Yocto Project Documentation 5.2", https://docs.yoctoproject.org, truy cập ngày 20/10/2026.

PHỤ LỤC

Phụ lục A. Build lại image và chạy kiểm thử

```
# máy chủ, trong thư mục repo; container yocto-build (Ubuntu 24.04) đã
có
git clone --branch yocto-5.2.4 --depth 1
https://git.yoctoproject.org/poky
./scripts/yocto.sh /workspace/scripts/setup-build.sh build-clean
./scripts/yocto.sh bitbake projectb-image
# ghi thẻ (kiểm SHA256 từng khối)
D=poky/build-clean/tmp/deploy/images/beaglebone-yocto
sudo scripts/flash-verified.sh \
   $D/projectb-image-beaglebone-yocto.rootfs.wic /dev/sdX
# trên bo: kết quả tự kiểm lúc khởi động
journalctl -u mpu6050-boot-check
# từ máy chủ: chép script và chạy demo
scripts/bo-setup.sh
ssh -t root@<bo> sh /tmp/demo.sh
```

Phụ lục B. Cấu trúc mã nguồn

```
driver/mpu6050/    mã driver, header uapi
apps/mpu6050-test/  chương trình kiểm thử
meta-projectb/     layer Yocto: image, wic, kernel, overlay
scripts/          build, ghi thẻ, kiểm thử, phân tích LA
docs/             tiến độ, kiến trúc, boot flow, ảnh LA, log
```

Phụ lục C. Tiến độ thực hiện thực tế

Bảng PL.1. Tiến độ thực hiện thực tế

Tuần	Nội dung
1	Môi trường build Docker, core-image-minimal, ghi thẻ có kiểm tra, khởi động tới login
2	Layer và image riêng, SSH bằng khóa, systemd, extlinux trong layer, vô hiệu bootloader eMMC
3	Màn hình HDMI, MPU6050 trên I2C2, overlay, kernel PREEMPT_RT; tag bringup-done
4	Kernel debug (lockdep), recipe module, probe, reset, cấu hình chip, đọc burst
5	Header uapi, /dev/mpu6050, kfifo, sysfs, chương trình kiểm thử
6	Ngắt DATA_RDY, threaded IRQ
7	FIFO theo lô, timestamp từng mẫu, tràn, I2C 400 kHz, kiểm thử RT và kernel thường
8	Đường lỗi, kref, watchdog, image nộp, tự kiểm khi khởi động, tài liệu, demo; tag v1.0-chuyennganh